
A PROBABILISTIC CHOREOGRAPHY LANGUAGE FOR PRISM

MARCO CARBONE ^a AND ADELE VESCHETTI ^b

^a Computer Science Department, IT University of Copenhagen, Rued Langgaards Vej 7, 2300 Copenhagen S, Denmark
e-mail address: carbonem@itu.dk, maca@itu.dk

^b Department of Computer Science, TU Darmstadt, Hochschulstraße 10, 64289 Darmstadt, Germany
e-mail address: adele.veschetti@tu-darmstadt.de

ABSTRACT. We present a choreographic framework for modelling and analysing concurrent probabilistic systems based on the PRISM model-checker. This is achieved through the development of a choreography language, which is a specification language that allows to describe the desired interactions within a concurrent system from a global viewpoint. Using choreographies gives a clear and complete view of system interactions, making it easier to understand the process flow and identify potential errors, which helps ensure correct execution and improves system reliability. We equip our language with a probabilistic semantics and then define a formal encoding into the PRISM language and discuss its correctness. Properties of programs written in our choreographic language can be model-checked by the PRISM model-checker via their translation into the PRISM language. Finally, we implement a compiler for our language and demonstrate its practical applicability via examples drawn from the use cases featured in the PRISM website.

1. INTRODUCTION

Programming distributed systems presents significant challenges due to their inherent complexity, and the possibility of obscure edge cases arising from their complex interactions. Unlike monolithic systems, distributed programs involve multiple nodes operating concurrently and communicating over networks, introducing a multitude of potential failure scenarios and nondeterministic behaviours. One of the primary challenges in understanding distributed systems lies in the fact that the interactions between multiple components can diverge from the sum of their individual behaviours. This emergent behaviour often results from subtle interactions between nodes, making it difficult to predict and reason about a system's overall behaviour.

PRISM [2] is a probabilistic model checker that offers a specialised language for the specification and verification of probabilistic concurrent systems. PRISM has been used in various fields, including multimedia protocols [21], randomised distributed algorithms [18, 20], security protocols [24, 19], and biological systems [12, 15]. At its core, PRISM provides a declarative language with a set of constructs for describing probabilistic behaviours and properties within a system. Given a distributed system, we can use PRISM to model the

Key words and phrases: optional comma separated list of keywords.

behaviour of each of its nodes, and then verify desired properties for the entire system. However, this approach can become difficult to manage as the number of nodes increases.

Choreographic programming [22] is an emerging programming paradigm in which programs, referred to as choreographies, serve as specifications providing a global perspective on the communication patterns inherent in a distributed system. In particular, instead of relying on a central orchestrator or controller to dictate the behaviour of individual components, choreographic languages focus on defining communication patterns and protocols that govern the interactions between entities. In essence, choreographies abstract away the internal details of individual components and emphasise the global behaviour as a composition of decentralised interactions. Although, this approach can be used for a proper subset of distributed systems, choreographies have proven to be very useful in many use cases since they facilitate the automatic generation of decentralised implementations that are inherently correct-by-construction [9].

This paper presents a choreographic language designed for modelling concurrent probabilistic systems. Additionally, we introduce a compiler for translating protocols described in this language into PRISM code. This choreographic approach not only simplifies the modelling process but also ensures integration with PRISM's powerful analysis capabilities.

To illustrate the idea with concrete commands, consider the following example. We have a choreographic language specification:

$$C \stackrel{\text{def}}{=} \mathbf{p} \rightarrow \{\mathbf{q}\} \begin{cases} \lambda_1 : (x' = 1) \ \& \ (y' = 2); \ C \\ \lambda_2 : (x' = 3) \ \& \ (y' = 1); \ C \end{cases}$$

This specification recursively defines an interaction where process $\mathbf{p}$ communicates with process $\mathbf{q}$, choosing either branch λ_1 (setting x to 1 and y to 2) or branch λ_2 (setting x to 3 and y to 1). The projection mechanism translates this global choreography into local PRISM modules by annotating each interaction with a unique label (e.g., a for λ_1 and b for λ_2) and using a state counter (e.g., $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$) to uniquely identify each step. The corresponding PRISM translation is:

$$\begin{array}{ll} \mathbf{p} : \{ & [a] \ s_{\mathbf{p}} = 0 \rightarrow \mu_1 : (x' = 1) \ \& \ (s'_{\mathbf{p}} = 1) \quad \square \ s_{\mathbf{p}} = 1 \rightarrow 1 : (s'_{\mathbf{p}} = 0) \\ & [b] \ s_{\mathbf{p}} = 0 \rightarrow \mu_2 : (x' = 3) \ \& \ (s'_{\mathbf{p}} = 2) \quad \square \ s_{\mathbf{p}} = 2 \rightarrow 1 : (s'_{\mathbf{p}} = 0) \quad \} \\ \mathbf{q} : \{ & [a] \ s_{\mathbf{q}} = 0 \rightarrow \gamma_1 : (y' = 2) \ \& \ (s'_{\mathbf{q}} = 1) \quad \square \ s_{\mathbf{q}} = 1 \rightarrow 1 : (s'_{\mathbf{q}} = 0) \\ & [b] \ s_{\mathbf{q}} = 0 \rightarrow \gamma_2 : (y' = 1) \ \& \ (s'_{\mathbf{q}} = 2) \quad \square \ s_{\mathbf{q}} = 2 \rightarrow 1 : (s'_{\mathbf{q}} = 0) \quad \} \end{array}$$

In this translation, the command labeled a in $\mathbf{p}$'s module corresponds to branch λ_1 (with rate μ_1), and the matching command in $\mathbf{q}$'s module (with rate γ_1) sets y to 2; similarly, label b corresponds to branch λ_2 (with rates μ_2 for $\mathbf{p}$ and γ_2 for $\mathbf{q}$), setting x and y appropriately. After executing an interaction command in each module, a subsequent reset command (with rate 1) sets the state counters $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$ to 0 due to the recursive call following the branching. This mechanism ensures that these commands are only executable when the system has reached this particular state.

Through our contributions, we aim to provide a smooth workflow for modeling, analyzing, and verifying concurrent probabilistic systems, ultimately increasing their usability in various application domains. In particular, by employing choreographies, we gain a clear and comprehensive view of the interactions occurring within the system, allowing us to discern the flow of processes and detect any potential sources of error in the modelling phase.

Contributions and Overview. We summarise our contributions as follows:

- we propose a choreographic language equipped with well-defined syntax and semantics, tailored specifically for describing concurrent systems with probabilistic behaviours (§ 3). To the best of our knowledge, this is the first probabilistic choreography language that is not a type abstraction;
- we introduce a semantics for the minimal fragment of PRISM needed for code generation from choreographies (§ 4), which follows PRISM’s original semantics [2];
- we establish a rigorous definition for a translation function from choreographies to PRISM (§ 5), and address its correctness. This translation serves as a crucial intermediary step in transforming models described in our choreographic language into PRISM-compatible representations;
- we give a compiler implementation that executes the defined translation function (§ 6), enabling users to utilise PRISM’s robust analysis features while benefiting from the expressiveness of choreographies;
- we show some use cases in order to demonstrate the applicability of our language (§ 7).

Changes with respect to the conference version. This article is an expanded version of our paper presented at COORDINATION 2024 [10]. It includes detailed definitions and proofs, which were previously omitted. Additionally, we provide a thorough analysis of richer language constructs incorporated in the implementation. Furthermore, we introduce a new set of use cases, analysed through choreographies, including an example that illustrates when the PRISM endpoint setting can be just as effective as a choreography-based approach.

2. A MOTIVATING EXAMPLE

As an example, we consider a simplified version of the thinkteam example, a three-tier data management system [25], presented in the PRISM documentation¹. The protocol aims to manage exclusive access to a single file, controlled by the `CheckOut` process. Users can request access, which can be granted based on the file’s current status. The goal is to ensure that only one user possesses the file at any given time while allowing for efficient access requests and retries in case of denial. Users move between different states (0, 1, or 2) based on the granted or denied access to the file with corresponding rates (λ, μ, θ); the `CheckOut` process transitions between two states (0 or 1):

```

1      ctmc
2      module User
3          User.STATE : [0..2] init 0;
4
5          [alpha.1] (User.STATE=0) → lambda : (User.STATE'=1);
6          [alpha.2] (User.STATE=0) → lambda : (User.STATE'=2);
7          [beta] (User.STATE=1) → mu : (User.STATE'=0);
8          [gamma.1] (User.STATE=2) → theta : (User.STATE'=1);
9          [gamma.2] (User.STATE=2) → theta : (User.STATE'=2);
10     endmodule
11
12     module CheckOut
13         CheckOut.STATE : [0..1] init 0;
14
15         [alpha.1,alpha.2] (CheckOut.STATE=0) → 1 : (CheckOut.STATE'=1);

```

¹<https://www.prismmodelchecker.org/casestudies/thinkteam.php>

```

16         [beta] (CheckOut.STATE=1) → 1 : (CheckOut.STATE'=0);
17         [gamma.1,gamma.2] (CheckOut.STATE=1) → 1 : (CheckOut.STATE'=1);
18     endmodule

```

In PRISM, modules are individual processes whose behaviour is specified by a collection of commands, in a declarative fashion. Processes have a local state, can interact with other modules and query each other's state. Above, the modules `User` (lines 2-10) and `CheckOut` (lines 12-18) can synchronise on different labels, e.g., `alpha.1`. On line 5, $(\text{User.STATE}=0 \hookrightarrow)$ is a condition indicating that this transition is enabled when `User.STATE` has value 0. The variable `lambda` is a rate, since the program models a Continuous Time Markov Chain (CTMC). The command $(\text{User.STATE}'=1)$ is an update, indicating that `User.STATE` changes to 1 when this transition fires.

Understanding the interactions between processes in this example might indeed be challenging, especially without additional context or explanation. Alternatively, when formalised using our choreographic language, the same model becomes significantly clearer.

```

C0 := CheckOut → User : (+["1*lambda"] ; C1 +["1*lambda"] ; C2)
C1 := CheckOut → User : (+["1*theta"] ; C0)
C2 := CheckOut → User : (+["1*mu"] ; C1 +["1*mu"] ; C2)

```

In this model, we define three distinct choreographies, namely `C0`, `C1`, and `C2`. These choreographies describe the interaction patterns between the modules `CheckOut` and `User`. The state updates resulting from these interactions are not explicitly depicted as they are not relevant for this particular protocol, but necessary in the PRISM implementation. As evident from this example, the choreographic language facilitates a straightforward understanding of the interactions between processes, minimizing the likelihood of errors. In fact, we can think of the choreography representation of this example as the product of the two PRISM modules seen above.

To formally analyze the system, we need to project the choreography onto individual components, resulting in a PRISM model that captures the behavior of each process explicitly. The projection process ensures that each role in the choreography, such as `User` and `CheckOut`, is assigned a state variable that tracks its execution progress. Interactions in the choreography correspond to PRISM synchronization labels, ensuring consistency across transitions. Furthermore, the rates in the choreography, such as λ, μ, θ , translate into transition rates in the PRISM model.

Each choreographic rule maps to a corresponding PRISM command. For instance, the choreographic transition:

```

C0 := CheckOut → User : (+["1*lambda"] ; C1)

```

projects to PRISM as $[\text{alpha.1}] (\text{User.STATE}=0) \rightarrow \text{lambda} : (\text{User.STATE}'=1)$; for the module `User` and as $[\text{alpha.1}] (\text{CheckOut.STATE}=0) \rightarrow 1 : (\text{CheckOut.STATE}'=1)$; for the `CheckOut`. This ensures both modules execute in sync under the label `alpha.1` and only when the modules are in the correct states (`CheckOut.STATE=0` and `User.STATE=0`). Similar transformations apply to other interactions.

3. CHOREOGRAPHY LANGUAGE

In the previous section, we showed our choreography language via an example. We now formalise our language by giving its syntax and semantics. For the sake of clarity, we slightly

change the syntax with respect to the syntax of our tool, adopting a more process-algebra format.

3.1. Syntax. Let $\mathbf{p}$ range over a (possibly infinite) set of module names $\mathcal{R}$, x over a (possibly infinite) set of variables $\mathbf{Var}$, and v over a (possibly infinite) set of values $\mathbf{Val}$. Choreographies, the key component of our language, are defined by the following syntax:

$$\begin{array}{lll}
C & ::= & \mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j \quad (\text{interaction}) \\
& | & \text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2 \quad (\text{conditional}) \\
& | & X \quad (\text{recursive call}) \\
& | & \mathbf{0} \quad (\text{inact}) \\
\mathcal{D} & ::= & X \stackrel{\text{def}}{=} C, \mathcal{D} \quad | \quad \emptyset \quad (\text{definitions}) \\
u & ::= & (x' = E) \ \& \ u \quad | \quad (x' = E) \quad (\text{assignments}) \\
E, g & ::= & f(\tilde{E}) \quad | \quad x \quad | \quad v \quad (\text{expressions})
\end{array}$$

The syntactic category C denotes choreographic programs. The interaction term $\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma\{\lambda_j : x_j = E_j. C_j\}_{j \in J}$ denotes an interaction initiated by module $\mathbf{p}$ with modules $\mathbf{p}_i$'s (for $\mathbf{p}$ and all $\mathbf{p}_i$ distinct). A choreography specifies what interaction must be executed next, shifting the focus from what can happen to what must happen. When the interaction happens, one of the j branches is selected as a continuation. Branching is a random move: the number $\lambda_j \in \mathbb{R}$ denotes either a probability or a rate. This will depend on the language we wish to use. In the case of probabilities, it must be the case that $0 \leq \lambda_j \leq 1$ and $\Sigma_j \lambda_j = 1$. Once a branch j is taken, the choreography will execute some assignments u_j . A single assignment has the syntax $(x' = E)$ meaning that the value obtained by evaluating expression E is assigned to variable x ; assignments can be concatenated with the operator $\&$. Note that x' is used for an assignment to x : here, we follow the syntax adopted in PRISM (see § 4). Expressions are obtained by applying some unspecified functions to other expressions or, as base terms, i.e., variables and values (denoted by v).

The term $\text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2$ denotes a system where module $\mathbf{p}$ evaluates the guard E (which can contain variables located at other modules) and then (deterministically) branches accordingly. The term X is a (possibly recursive) procedure call: in the semantics, we assume that such procedure names are defined separately. The term $\mathbf{0}$ denotes the system finishing its computation.

3.2. Semantics. The semantics of a choreography is a relation that captures how the values assigned to variables are modified when the various modules synchronise with each other. Similar to the operational semantics of imperative languages, we define a state, denoted by S , as a mapping from variables to values, i.e., $S : \mathbf{Var} \rightarrow \mathbf{Val}$.

Given a state, substitution allows to modify some of its values:

Definition 1. *Given a value v and a variable x , a substitution $[v/x]$ is an update on a state, i.e., $S[v/x](y) = \begin{cases} v & \text{if } y = x \\ S(y) & \text{otherwise} \end{cases}$. Then, the update $S[u]$ is such that $S[x' = E \ \& \ u] = S[E \downarrow_S / x][u]$ and $S[x' = E] = S[E \downarrow_S / x]$, where $E \downarrow_S$ is an unspecified (decidable) evaluation of the expression E in the state S .*

Given the set of all possible states $\mathcal{S}$ and a set of definitions $\mathcal{D}$ of the form $X \stackrel{\text{def}}{=} C$, we can define the operational semantics of choreographies as the minimal relation $\longrightarrow^{\mathcal{D}} \subseteq \mathcal{S} \times C \times \mathbb{R} \times \mathcal{S} \times C$ such that (we omit $\mathcal{D}$ if not relevant):

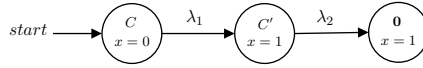
$$\begin{aligned}
(\text{Interact}) \quad & (S, p \rightarrow \{p_1, \dots, p_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j) \longrightarrow_{\lambda_j} (S[u_j], C_j) \\
(\text{IfThenElseT}) \quad & E \downarrow_S = \text{tt} \Rightarrow (S, \text{if } E @ p \text{ then } C_1 \text{ else } C_2) \longrightarrow_1 (S, C_1) \\
(\text{IfThenElseF}) \quad & E \downarrow_S = \text{ff} \Rightarrow (S, \text{if } E @ p \text{ then } C_1 \text{ else } C_2) \longrightarrow_1 (S, C_2) \\
(\text{Call}) \quad & X \stackrel{\text{def}}{=} C \in \mathcal{D} \Rightarrow (S, X) \longrightarrow_1 (S, C)
\end{aligned}$$

The transition relation is a Discrete Time Markov Chain (DTMC) or a Continuous Time Markov Chain (CTMC) depending on whether we use probabilities or rates in the branching construct. Note that states of the Markov chain are the pairs (S, C) , while the transitions are given by the relation $\longrightarrow$.

Example 1. Consider the following choreography:

$$C = p \rightarrow \{q\} \lambda_1 : (x' = 1); \quad p \rightarrow \{q\} \lambda_2 : (x' = 1); \mathbf{0}$$

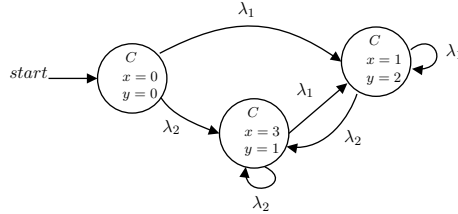
The semantics of C starting from a state in which $S(x) = S(y) = 0$ can be depicted as follows (for $C' = p \rightarrow \{q\} \lambda_1 : (x' = 1); \mathbf{0}$):



Example 2. Consider the following definition:

$$C \stackrel{\text{def}}{=} p \rightarrow \{q\} \left\{ \begin{array}{l} \lambda_1 : (x' = 1) \& (y' = 2); C \\ \lambda_2 : (x' = 3) \& (y' = 1); C \end{array} \right.$$

The semantics of C starting from a state in which $S(x) = S(y) = 0$ can be depicted as follows:



3.3. Other language constructs. Our language includes constructs that are not part of the formal syntax defined above. These constructs are purely syntactic sugar and can be easily encoded. Below, we discuss each of them:

- *Parametric modules.* In our implemented language, modules can be parameterised (indexed) as done in PRISM. We denote parameterised roles as $p[n]$ for n ranging some finite set N . As an example, the choreography

$$p[i] \rightarrow \{q[i]\} : \lambda : U; \quad r \rightarrow \{q[i]\} : \lambda : U; \quad \mathbf{0}$$

can be easily encoded as:

$$\begin{aligned} p1 &\rightarrow \{q1\} : \lambda : U; \\ p2 &\rightarrow \{q2\} : \lambda : U; \\ p3 &\rightarrow \{q3\} : \lambda : U; \\ r &\rightarrow \{q1\} : \lambda : U; \\ r &\rightarrow \{q2\} : \lambda : U; \\ r &\rightarrow \{q3\} : \lambda : U; \quad \mathbf{0} \end{aligned}$$

Additionally, the choreography

$$p[i] \rightarrow \{q[i]\} : \lambda : U; \quad q[i+1] \rightarrow \{p[i]\} : \lambda : U; \quad \mathbf{0}$$

can be encoded in our model language as

$$\begin{aligned} p1 &\rightarrow \{q1\} : \lambda : U; \\ p2 &\rightarrow \{q2\} : \lambda : U; \\ p3 &\rightarrow \{q3\} : \lambda : U; \\ q2 &\rightarrow \{p1\} : \lambda : U; \\ q3 &\rightarrow \{p2\} : \lambda : U; \\ q1 &\rightarrow \{p3\} : \lambda : U; \quad \mathbf{0} \end{aligned}$$

Given that parameters range over a finite set, this operation is redundant as far as the theory is concerned.

- *The foreach construct.* The use of parametric modules can be further facilitated by introducing syntactic sugar that allows iteration over the set of indices that parameterise these modules. To this end, our language implementation includes the construct **foreach** $(k \text{ op } i) \ u@A[i]$, which can be used in front of and update u to parameterise with respect to multiple instances of the same type of variable over different modules. The construct enables the expression $(k \text{ op } i)$ to range over a set of indices, which can then be used in $u@A[i]$. In essence, it provides a more concise and readable way to express operations over multiple indexed modules. Notably, this construct can be encoded explicitly by numbering the modules manually, provided that the indices are known at compile time rather than determined dynamically at runtime.
- *Non-deterministic Synchronisation.* Our implementation supports the non-deterministic synchronisation language construct **allsynch** $\{p_i : G_i\}_{i \in I}$, where G has the form $g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$. The core idea behind this construct is to enable a set of roles, denoted by p_i , to synchronise while allowing each role to non-deterministically select from a range of possible local actions. This provides a structured mechanism for defining interactions where multiple roles must coordinate, but their precise behaviour may vary dynamically (with a certain probability distribution/rate).

A key advantage of this construct is its compactness and readability in specifying this type of interactions. For instance, in a scenario where two roles, p and q , participate in a synchronised exchange, the **allsynch** syntax allows for a concise definition of conditions

under which each role updates its state. This avoids the need for manually encoding synchronisation through nested conditional constructs. Despite its expressiveness, the **allsynch** construct does not introduce a new semantics but serves as syntactic sugar for an equivalent formulation. In fact, **allsynch** can be rewritten using a series of nested **if-then-else** constructs, ensuring that the synchronisation conditions are met before proceeding with the interaction. For example, consider the non-deterministic synchronisation between roles **p** and **q**:

$$\text{allsynch} \left\{ \begin{array}{l} \text{p} : (x = 5) \rightarrow 10 : (x' = 0) \\ \text{p} : (x = 1) \rightarrow 5 : (x' = 100) \\ \text{q} : (y = 1) \rightarrow 1 : (y' = 0) \end{array} \right\}; 0$$

This construct provides a compact and structured way to define synchronized interactions. However, it can be rewritten without using the **allsynch** syntax by explicitly handling conditions using the **if-then-else** construct:

```

if (x = 5)@p then
  if (y = 1)@q then
    p → {q} 10 : (x' = 0)&(y' = 0); 0
  else 0
else
  if (x = 1)@p then
    if (y = 1)@q then
      p → {q} 5 : (x' = 100)&(y' = 0); 0
    else 0
  else 0

```

The two formulations are equivalent, with the **allsynch** syntax acting as syntactic sugar for a structured and readable representation of synchronization. The expanded version using **if-then-else** makes explicit the conditional execution of interactions but retains the same behaviour.

The **allsynch** construct is more efficient (in terms of code) than manually encoding synchronisation with nested **if-then-else** statements. As the number of synchronising roles and conditions increases, the depth and complexity of the nested conditionals grow exponentially, making the explicit formulation harder to read, write, and process. In contrast, **allsynch** provides a compact and structured way to express the same logic without the combinatorial explosion of nested conditionals.

4. THE PRISM LANGUAGE

We now give a formal definition of a fragment of the PRISM language by introducing its formal syntax and semantics.

4.1. Syntax. We reuse some of the syntactic terms used for our choreography language, including assignments and expressions. In the sequel, let a range over a (possibly infinite)

set of labels $\mathcal{L}$. We define the syntax of (a subset of) the PRISM language as follows:

(Networks)	$N, M ::=$	$\mathbf{0}$	empty network
		$ \mathbf{p} : \{F_i\}_i$	module
		$ M [A] M$	parallel composition
(Commands)	$F ::=$	$[\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$	$(\alpha \in \{\epsilon\} \cup \mathcal{L})$

Networks are the top syntactic category for system of modules composed together. The term $\mathbf{0}$ represent an empty network. A module is meant to represent a process running in the system and is denoted by its name and its commands, formally written as $\mathbf{p} : \{F_i\}_i$, where $\mathbf{p}$ is the name and the F_i 's are commands. Networks can be composed in parallel, in a CSP style: a term like $M_1|[A]|M_2$ says that networks M_1 and M_2 can synchronise using labels in the finite set A . In this work, we omit PRISM's hiding and substitution constructs as they are irrelevant for our current choreography language. Commands in a module have the form $[\alpha]g \rightarrow \Sigma_{i \in I} \{\lambda_i : u_i\}$. The character α can either be the empty string ϵ or a label a , i.e., $\alpha \in \{\epsilon\} \cup \mathcal{L}$. If ϵ then no synchronisation is required. On the other hand, if there is label a then there will be a synchronisation with other modules that must synchronise on a . The term g is a guard on the current variable state. If both label and guard are enabled, then the command executes a branch i with probability/rate λ_i . As for choreographies, if the λ_i 's are probabilities, we must have that $0 \leq \lambda_i \leq 1$ and $\Sigma_{i \in I} \lambda_i = 1$.

4.2. Semantics. To give a probabilistic semantics to the PRISM language, we follow the approach given in the PRISM documentation [2]. Hereby, we do that by defining two relations: one with labels for networks and one on states. Our relation on networks is the minimum relation $\rightsquigarrow$ satisfying the rules given in Fig. 1. Rule (M) just exposes a command

$$\begin{array}{c}
 \frac{F \in \{F_k\}_k}{\mathbf{p} : \{F_k\}_k \rightsquigarrow F} \text{ (M)} \quad \frac{\exists j \in \{1,2\}. M_j \rightsquigarrow [\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i \quad \alpha \notin A}{M_1|[A]|M_2 \rightsquigarrow [\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i} \text{ (P}_1\text{)} \\
 \\
 \frac{M_1 \rightsquigarrow [a] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i \quad M_2 \rightsquigarrow [a] g' \rightarrow \Sigma_{j \in J} \lambda'_j : u'_j \quad a \in A}{M_1|[A]|M_2 \rightsquigarrow [a] g \wedge g' \rightarrow \Sigma_{i,j} \lambda_i * \lambda'_j : u_i \& u'_j} \text{ (P}_2\text{)}
 \end{array}$$

Figure 1: Semantics for PRISM networks

at network level. Rule (P₁) propagates a command through parallel composition if α is empty or if the label a is not part of the set A . When the label a is in A , we apply rule (P₂). In this case, the product of the probabilities/rates must be taken by extending the two different branches to every possible combination. This also includes the combination of the associated assignments.

Based on the relation above, given $M \rightsquigarrow [\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$ and two states S and S' , we define the function

$$\mu([\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i, S, S') = \Sigma_{S[u_i]=S', i \in I} \lambda_i$$

which gives the probability/rate for the system to go from state S to state S' after executing command $[\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$, for some α . If the λ_i are probabilities, then the function must be a probability distribution. Note that $\mu(F, S, S')$ only denotes the probability/rate for the system to move from state S to state S' after executing command F . However, there

can be other commands derived from a given network M through the relation $\rightsquigarrow$ that would cause a transition from S to S' . Therefore, we define the transition relation on states $M \vdash S \rightarrow_\lambda S'$ as

$$\frac{\forall j, \alpha. M \rightsquigarrow F_j \quad S \vdash F_j}{M \vdash S \rightarrow_{\sum_j \mu(F_j, S, S')} S'} \text{ (Transition)}$$

where $S \vdash [\alpha] g \rightarrow \sum_{i \in I} \lambda_i : u_i$ is defined as $g \downarrow_S$. Note that since PRISM is declarative, a term M never changes while the state of the system evolves.

It is important to point out that, in general, the transition rule above does not give the exact probability of a transition in case of a Markov chain (DTMC), since the sum $\sum_j \mu(F_j, S, S')$ could be a value greater than 1. In order to get the right probability, the value has to be normalised for all reachable S' . In the next section, we will show that this is not an issue for networks that are obtained from our translation from choreography to PRISM.

Example 3. Consider the following network M :

$$\begin{aligned} \mathbf{p} : \{ & \quad \Box x = 0 \rightarrow 1 : (x' = 1) \\ & \quad [a] y < 1 \rightarrow 0.4 : (x' = x + 1) + 0.6 : (x' = x) \quad \} \\ \mathbf{q} : \{ & \quad \Box y = 0 \rightarrow 1 : (y' = 1) \\ & \quad [a] x < 1 \rightarrow 0.5 : (y' = y + 1) + 0.5 : (y' = y) \quad \} \end{aligned}$$

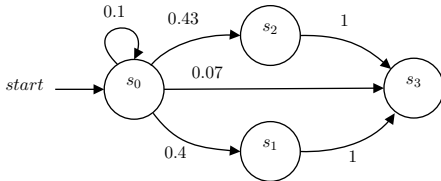
above, the two modules $\mathbf{p}$ and $\mathbf{q}$ can both do independent actions, as well as synchronising on label a . Applying the semantics, we can easily derive $M \rightsquigarrow \Box x = 0 \rightarrow 1 : (x' = 1)$, $M \rightsquigarrow \Box y = 0 \rightarrow 1 : (y' = 1)$, and $M \rightsquigarrow F$, such that

$$\begin{aligned} F = [a] x < 1 \ \& \ y < 1 \rightarrow & 0.2 : (x' = x + 1) \ \& \ (y' = y + 1) \\ & + 0.2 : (x' = x + 1) \ \& \ (y' = y) \\ & + 0.3 : (x' = x) \ \& \ (y' = y + 1) \\ & + 0.3 : (x' = x) \ \& \ (y' = y) \end{aligned}$$

Let $s_0 = (x = 0, y = 0)$, $s_1 = (x = 1, y = 0)$, $s_2 = (x = 0, y = 1)$, and $s_3 = (x = 1, y = 1)$ be all possible states, with s_0 a starting state. Then,

$$\begin{aligned} \mu(\Box x = 0 \rightarrow 1 : (x' = 1), s_0, s_1) &= 1 & \mu(F, s_0, s_1) &= 0.2 \\ \mu(\Box y = 0 \rightarrow 1 : (y' = 1), s_0, s_2) &= 1 & \mu(F, s_0, s_2) &= 0.3 \\ \mu(F, s_0, s_0) &= 0.3 & \mu(F, s_0, s_3) &= 0.2 \end{aligned}$$

Now, by (Transition), we have that $M \vdash s_0 \rightarrow_{1.2} s_1$, $M \vdash s_0 \rightarrow_{1.3} s_1$. Clearly, both transitions should be normalised, finally yielding the following DTMC:



Example 4. The choreography presented in Example 2 can be described by the following PRISM network M (for $\lambda_i = \mu_i * \gamma_i$):

$$\begin{aligned} \mathbf{p} : \{ & \quad [a] s_p = 0 \rightarrow \mu_1 : (x' = 1) \ \& \ (s'_p = 1) & \quad \Box s_p = 1 \rightarrow 1 : (s'_p = 0) \\ & \quad [b] s_p = 0 \rightarrow \mu_2 : (x' = 3) \ \& \ (s'_p = 2) & \quad \Box s_p = 2 \rightarrow 1 : (s'_p = 0) \quad \} \\ \mathbf{q} : \{ & \quad [a] s_q = 0 \rightarrow \gamma_1 : (y' = 2) \ \& \ (s'_q = 1) & \quad \Box s_q = 1 \rightarrow 1 : (s'_q = 0) \\ & \quad [b] s_q = 0 \rightarrow \gamma_2 : (y' = 1) \ \& \ (s'_q = 2) & \quad \Box s_q = 2 \rightarrow 1 : (s'_q = 0) \quad \} \end{aligned}$$

The two modules $\mathbf{p}$ and $\mathbf{q}$ synchronise on the labels a and b . Applying the semantics, the global state can evolve according to F_1 or F_2 , defined as follows:

$$\begin{aligned} F_1 = [a] \ s_{\mathbf{p}} = 0 \ \& \ s_{\mathbf{q}} = 0 \ \rightarrow \ \mu_1 * \gamma_1 : (x' = 1) \ \& \ (y' = 2) \ \& \ (s'_{\mathbf{p}} = 1) \ \& \ (s'_{\mathbf{q}} = 1) \\ F_2 = [b] \ s_{\mathbf{p}} = 0 \ \& \ s_{\mathbf{q}} = 0 \ \rightarrow \ \mu_2 * \gamma_2 : (x' = 3) \ \& \ (y' = 1) \ \& \ (s'_{\mathbf{p}} = 2) \ \& \ (s'_{\mathbf{q}} = 2) \end{aligned}$$

5. PROJECTION

In this section, we provide a rigorous treatment of projection, which constitutes the mapping from choreographies to the PRISM language.

5.1. Mapping Choreographies to PRISM. The process of generating endpoint code from a choreography is commonly referred to as *projection*. Typically, projection is defined separately for each module appearing in the choreography program, i.e., given a module (often called a role) and a choreography, it generates the code for that particular role. However, this is not the case in our approach, as PRISM relies solely on label synchronisation and a notion of state, which can be modified through standard imperative assignments enabled by conditions on the state. Thus, our approach simulates a choreography interaction in PRISM by using (i) labels on which each involved module can synchronise and (ii) the state to enable the correct commands at the appropriate times.

Before formalising this idea, we make a slight abuse of notation by assuming that each interaction in a choreography is annotated with a label. We refer to such a choreography as an *annotated choreography*:

Definition 2 (Annotated Choreography). *An annotated choreography is obtained from the choreography syntax by adding a label to each interaction:*

$$C ::= \mathbf{p} \rightarrow^a \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; \ C_j \mid \dots$$

The intuition behind annotations is that they allow us to identify a particular interaction in the choreography, enabling all modules involved to synchronise. Since these annotations must be unique, we make the following assumption, which is key to our results:

Assumption 1. *Each annotation in an annotated choreography occurs exactly once.*

As an example, the following choreography is annotated correctly:

$$\mathbf{p} \rightarrow^a \{\mathbf{q}, \mathbf{r}\} : \left(\begin{array}{l} \lambda_1 : \mathbf{0} \\ + \\ \lambda_2 : \text{if } E@_{\mathbf{p}} \text{ then } \mathbf{p} \rightarrow^b \{\mathbf{q}\} : \lambda_1 : \mathbf{0} \text{ else } \mathbf{0} \end{array} \right)$$

while, the one below is not:

$$\mathbf{p} \rightarrow^a \{\mathbf{q}, \mathbf{r}\} : \left(\begin{array}{l} \lambda_1 : \mathbf{0} \\ + \\ \lambda_2 : \text{if } E@_{\mathbf{p}} \text{ then } \mathbf{p} \rightarrow^a \{\mathbf{q}\} : \lambda_1 : \mathbf{0} \text{ else } \mathbf{0} \end{array} \right)$$

In order to ensure consistency for subsequent statements in a choreography, our definition of projection uses the function $\text{nodes}(C)$, which returns the number of nodes in abstract syntax tree of C . Formally, it is defined as:

$$\begin{aligned} \text{nodes}(\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; \ C_j) &= 1 + \sum_{j \in J} \text{nodes}(C_j) \\ \text{nodes}(\text{if } E@_{\mathbf{p}} \text{ then } C_1 \text{ else } C_2) &= 1 + \text{nodes}(C_1) + \text{nodes}(C_2) \\ \text{nodes}(X) &= \text{nodes}(\mathbf{0}) = 1 \end{aligned}$$

We now define the projection function. Since there are key differences between using probabilities and using rates, we proceed separately. We begin with choreographies that involve rates:

Definition 3 (Projection, CTMC). *Given an annotated choreography with rates C , a module p , a natural number ι , and $J = \{1, \dots, m\}$, we define the function proj as:*

$$\begin{aligned}
& \text{proj}(\mathbf{q}, p \rightarrow^a \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j, \iota) = \boxed{\text{if } \mathbf{q} = \mathbf{p}} \\
& \quad \left\{ [a_j] \ s_q = \iota \rightarrow \lambda_j : s_q = s_q + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k) \ \& \ u_j \downarrow_q \right\}_{j \in J} \\
& \quad \cup \bigcup_j \text{proj}(\mathbf{q}, C_j, \iota + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k)) \\
& \text{proj}(\mathbf{q}, p \rightarrow^a \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j, \iota) = \boxed{\text{if } \mathbf{q} \in \{\mathbf{p}_1, \dots, \mathbf{p}_n\}} \\
& \quad \left\{ [a_j] \ s_q = \iota \rightarrow 1 : s_q = s_q + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k) \ \& \ u_j \downarrow_q \right\}_{j \in J} \\
& \quad \cup \bigcup_j \text{proj}(\mathbf{q}, C_j, \iota + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k)) \\
& \text{proj}(\mathbf{q}, p \rightarrow^a \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j, \iota) = \boxed{\text{if } \mathbf{q} \notin \{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\}} \\
& \quad \bigcup_j \text{proj}(\mathbf{q}, C_j, \iota + \sum_{k=1}^{j-1} \text{nodes}(C_k)) \\
& \text{proj}(\mathbf{q}, \text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2, \iota) = \boxed{\text{if } \mathbf{q} = \mathbf{p}} \\
& \quad \left\{ \begin{array}{l} \boxed{\ s_q = \iota \ \& \ E \rightarrow 1 : s'_q = \iota + 1,} \\ \boxed{\ s_q = \iota \ \& \ \text{not}(E) \rightarrow 1 : s'_q = \iota + \text{nodes}(C_1) + 1} \end{array} \right\} \\
& \quad \cup \text{proj}(\mathbf{p}, C_1, \iota + 1) \cup \text{proj}(\mathbf{p}, C_2, \iota + \text{nodes}(C_1) + 1) \\
& \text{proj}(\mathbf{q}, \text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2, \iota) = \boxed{\text{if } \mathbf{q} \neq \mathbf{p}} \\
& \quad \text{proj}(\mathbf{q}, C_1, \iota) \cup \text{proj}(\mathbf{q}, C_2, \iota + \text{nodes}(C_1)) \\
& \text{proj}(\mathbf{q}, \mathbf{0}, \iota) = \emptyset \\
& \text{proj}(\mathbf{q}, X, \iota) = \boxed{\ s_q = \iota \rightarrow 1 : s'_q = \iota' \quad \text{where } \text{defs}(X) = \iota' }
\end{aligned}$$

We examine the various cases in the definition above. The first three cases deal with the projection of an interaction. When projecting the first module p , we create one command $[a_j] \ s_q = \iota \rightarrow \lambda_j : s_q = s_q + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k) \ \& \ u_j \downarrow_q$ for each branch such that

- the label a_j and its uniqueness ensure that all modules take the same branch;
- the guard $s_q = \iota$ ensures that these commands are only executable when the system has reached this particular state, identified by the reserved variable s_q ;
- the rate λ_j is the rate that appears in the same branch of the choreography
- the successor state is determined by incrementing s_q , depending on which branch was selected—the function nodes ensures that every interaction in all branches is assigned to a different counter value, thereby also discarding all branches that are not selected;
- the projected update $u_j \downarrow_q$ acts as a filter on the list of updates in u_j , ensuring that only those variables local to q are updated.

Note that the translation works only with rates. In the case of probabilities, the definition above is incorrect, as we must ensure that the probabilities in a branching sum to 1.

The second case defines the projection of an interaction for one of the modules $\{p_1, \dots, p_n\}$. Similarly to the previous case, we define a command for each branch of the interaction. However, the rate of each command is set to 1, ensuring that each branch synchronises with probability $\lambda_j \cdot 1$ (see rule (P₂) in Figure 1). The third case is the one when we are projecting a module that is not in the set $\{p, p_1, \dots, p_n\}$. The if-then-else construct focuses on the module p where the guard E must be evaluated. As a consequence, we do not need to have any label for synchronisation. For recursive calls, we generate a command that resets the counter to a distinct value given by the auxiliary function `defs`.

Example 5. *In order to show how our projection works, consider the following example in which we apply the projection to the choreography in Example 2 and obtain the PRISM modules from Example 4. In Example 2, we defined a recursive choreography in which role p interacts with role q through two branches. Its annotated form can be written as:*

$$C \stackrel{\text{def}}{=} p \rightarrow^a \{q\} \left\{ \begin{array}{l} \lambda_1 : (x' = 1) \& (y' = 2); C \\ \lambda_2 : (x' = 3) \& (y' = 1); C \end{array} \right.$$

From label a , each branch of the choreography is identified by a unique label, say a_1 for the first branch and a_2 for the second. Then, the state of each module can be tracked by counters s_p and s_q . The various steps of the projection can then be summarised as follows:

- (1) **Computing the States.** Starting from the initial counter values $s_p = 0$ and $s_q = 0$, we apply our projection function to determine the new state values for each interaction. The auxiliary function `nodes(C)` counts the steps within each branch of the choreography. Since each branch in Example 2 consists of a single interaction followed by a recursive call to C , we can compute the state updates as follows. In particular, for the j^{th} branch, the new state is given by $\iota + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k)$. Applying this formula to our example:

(a) Branch λ_1 (label a_1):

- The initial state is $s_p = 0$ and $s_q = 0$.
- in the first branch ($j = 1$), the sum $\sum_{k=1}^{j-1} \text{nodes}(C_k)$ is equal to 0, therefore

$$s'_p = 0 + 1 = 1, \quad s'_q = 0 + 1 = 1$$

- The update rule for this branch is $s'_p = 1 \quad \& \quad s'_q = 1$

(b) Branch λ_2 (label a_2):

- Again, starting from $s_p = 0$ and $s_q = 0$.
- in the second branch ($j = 2$), the sum $\sum_{k=1}^{j-1} \text{nodes}(C_k)$ is equal to 1 because the first branch contains only the recursive call to C . Hence,

$$s'_p = 0 + 2 = 2, \quad s'_q = 0 + 2 = 2$$

- The update rule for this branch is $s'_p = 2 \quad \& \quad s'_q = 2$

Thus, each branch is assigned a unique state to ensure that only one transition can be taken at a time in the PRISM model. The recursive nature of the choreography ensures that the state counters return to 0 after each interaction, allowing the process to repeat.

- (2) **Assigning Unique Labels.** For each branch, a unique label is generated from the interaction's base label. In this example, the first branch is assigned label a_1 and the second a_2 . These labels serve as synchronisation points between the interacting modules. In our projection, role p (the initiator) uses the corresponding rates (e.g. μ_1 and μ_2 for branches 1 and 2) while role q uses a fixed rate (equal to 1) for synchronisation.
- (3) **From Choreography to PRISM.** As detailed in Example 4, the projected PRISM network is obtained by creating commands for each branch in both roles. Each command

is guarded by a condition on the state counter (for instance, $s_p = 0$ for the first branch) and includes the update that sets the counter to the new state computed above. The modules for $\mathbf{p}$ and $\mathbf{q}$ synchronize on the unique labels a and b , and the overall system's global transitions (e.g. F_1 and F_2) are derived by the composition of these synchronized commands. The rates of these transitions are computed as the product of the individual rates (i.e., $\lambda_i = \mu_i * \gamma_i$).

As hinted above, the projection in Definition 3 would be incorrect if instead of using rates we used probabilities. This is simply because we cannot force both $\mathbf{p}$ and $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$ to take the same branch with the probability distribution of the λ_i 's. To fix this problem, we have the following definition instead:

Definition 4 (Projection, DTMC). *Given an annotated choreography with probabilities C , a module $\mathbf{p}$, and a natural number ι , we define proj as:*

$$\begin{aligned} \text{proj}(\mathbf{q}, \mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j, \iota) = & \boxed{\text{if } \mathbf{q} = \mathbf{p}} \\ & \left\{ \begin{array}{l} \square s_q = \iota \rightarrow \Sigma_{j \in J} \lambda_j : s'_q = \iota + 1 + j, \\ \{[l_j] s_q = \iota + 1 + j \rightarrow 1 : s'_q = \iota + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k) \ \& \ u_j \downarrow_q\}_{j \in J} \end{array} \right\} \\ \cup \bigcup_j \text{proj}(\mathbf{q}, C_j, s + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k)) \end{aligned}$$

The other cases of the definition are equivalent to those in Definition 3.

The fix is immediate: module $\mathbf{p}$ takes a (probabilistic) internal decision on the j^{th} branch and then synchronises on label l_j with $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$.

5.2. Correctness. Our projection operations are correct with respect to the semantics of choreographies and PRISM. In order to state our main result, we need to use the notions of *head modules* and *strongly connected* choreography. The former identifies the modules involved in the next action of a choreography:

Definition 5 (Head Modules). *The function hMods is defined as follows:*

$$\begin{aligned} \text{hMods}(\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j) &= \{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\} \\ \text{hMods}(\text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2) &= \{\mathbf{p}\} \\ \text{hMods}(X) &= \text{hMods}(C) & (\text{if } X \stackrel{\text{def}}{=} C \in \mathcal{D}) \\ \text{hMods}(\mathbf{0}) &= \emptyset \end{aligned}$$

Then, the property of strongly connected is defined below.

Definition 6 (Strongly Connected Choreography). *A choreography C is strongly connected, written $\text{sConn}(C)$, if it satisfies the following conditions:*

$$\begin{aligned} & \frac{\forall j \in J. \text{sConn}(C_j) \wedge \text{hMods}(C_j) \neq \emptyset \Rightarrow \text{hMods}(C_j) \cap \{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\} \neq \emptyset}{\text{sConn}(\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j)} \\ & \frac{\forall j \in \{1, 2\}. \text{sConn}(C_j) \wedge \text{hMods}(C_j) \neq \emptyset \Rightarrow \mathbf{p} \in \text{hMods}(C_j)}{\text{sConn}(\text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2)} \\ & \frac{\text{sConn}(C) \quad X \stackrel{\text{def}}{=} C \in \mathcal{D}}{\text{sConn}(X)} \quad \frac{}{\text{sConn}(\mathbf{0})} \end{aligned}$$

The notion of connectedness is quite well-known in the literature. Since our framework is based on synchronous communication, we follow the same approach as that of Carbone et al. [8]. The basic idea is that each interaction shares at least one module with the subsequent choreography. In particular, in every branch of a probabilistic choice or an if-then-else involving modules $\mathbf{p}_1, \dots, \mathbf{p}_n$, the first action (if any) of every other module $\mathbf{q}$ must be an interaction with one of $\mathbf{p}_1, \dots, \mathbf{p}_n$, possibly after unfolding recursive calls. For example, the choreography

$$X \stackrel{\text{def}}{=} \mathbf{p} \rightarrow \{\mathbf{q}\} : \left(\begin{array}{l} \lambda_1 : u_1; \mathbf{q} \rightarrow \{\mathbf{r}\} : \lambda'_1 : u'_1; X \\ \lambda_2 : u_2; X \end{array} \right)$$

is strongly connected while $\mathbf{p} \rightarrow \{\mathbf{q}\} : (\lambda_1 : u_1; \mathbf{r}_1 \rightarrow \{\mathbf{r}_2\} : \lambda'_1 : u'_1; \mathbf{0}.)$ is not.

We are now ready for our main theorem. In the sequel, S_+ is obtained from state S by extending its domain with the extra variables $s_{\mathbf{q}}$ (one for each module in a choreography) used by the projection. In the projection, we utilize alphabetized parallel composition $\parallel$, wherein modules synchronise solely on labels that appear in both modules.

Theorem 1 (Projection). *Given a choreography C such that $\text{sConn}(C)$, we have that $(S, C) \longrightarrow_{\lambda} (S', C')$ if and only if $\parallel_{\mathbf{q} \in C} \text{proj}(\mathbf{q}, C, \iota) \vdash S_+ \longrightarrow_{\lambda} S'_+$.*

Proof. The proof proceeds by cases on the syntax of C .

- $C = \mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \sum_{j \in J} \lambda_j : u_j; C_j$. By (the only applicable) rule (Interact), we have that

$$(S, \mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \sum_{j \in J} \lambda_j : u_j; C_j) \longrightarrow_{\lambda_j} (S[u_j], C_j)$$

By definition of projection, we obtain the following PRISM commands. Role $\mathbf{p}$ is projected as:

$$\left\{ [a_j] \ s_{\mathbf{p}} = \iota \rightarrow \lambda_j : s_{\mathbf{p}} = s_{\mathbf{p}} + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k) \ \& \ u_j \downarrow_{\mathbf{p}} \right\}_{j \in J} \cup \bigcup_j \text{proj}(\mathbf{q}, C_j, \iota + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k))$$

Roles $\mathbf{p}_i$ are projected as:

$$\left\{ [a_j] \ s_{\mathbf{p}_i} = \iota \rightarrow 1 : s_{\mathbf{p}_i} = s_{\mathbf{p}_i} + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k) \ \& \ u_j \downarrow_{\mathbf{p}_i} \right\}_{j \in J} \cup \bigcup_j \text{proj}(\mathbf{p}_i, C_j, \iota + 1 + \sum_{k=1}^{j-1} \text{nodes}(C_k))$$

while any other role is projected as:

$$\bigcup_j \text{proj}(\mathbf{q}, C_j, \iota + \sum_{k=1}^{j-1} \text{nodes}(C_k))$$

We need to show two things: first, that the projection above can make the same transition; second, that if the projection makes a transition, it must be corresponding to that of the choreography above. Observe that the state of a generated CTMC is uniquely identified by the counter ι . The uniqueness of the label a (Assumption 1) makes sure that all and only those modules involved in this interaction synchronise with this action (this shows from the rules in Figure 1). As a consequence of this and since choreographies are strongly connected, the commands generated by this step of the translation are such that any state S_+ is exclusively going to enable these commands (because of the guard $s_{\mathbf{q}} = \iota$) which obviously implies that it must be done with rate λ_j applying the rules in Figure 1. This argument is key for proving both directions of the if and only if.

- $C = \text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2$. In this case, the projection of $\mathbf{p}$ is

$$\left\{ \begin{array}{l} \square s_{\mathbf{p}} = \iota \ \& \ E \rightarrow 1 : s'_{\mathbf{p}} = \iota + 1, \\ \square s_{\mathbf{p}} = \iota \ \& \ \text{not}(E) \rightarrow 1 : s'_{\mathbf{p}} = \iota + \text{nodes}(C_1) + 1 \end{array} \right\} \cup$$

$$\text{proj}(\mathbf{p}, C_1, \iota + 1) \cup \text{proj}(\mathbf{p}, C_2, \iota + \text{nodes}(C_1) + 1)$$

while, for all other roles, we have

$$\text{proj}(\mathbf{q}, C_1, \iota) \cup \text{proj}(\mathbf{q}, C_2, \iota + \text{nodes}(C_1))$$

In this case, role $\mathbf{p}$ is enabled by the counter. All other roles are not enabled simply because we assume that the choreography is strongly connected; hence, any other synchronisation or if-then-else statement is blocked, as it must involve $\mathbf{p}$ (on different counter value).

- $C = X$, and $C = \mathbf{0}$. Similar to the other case.

The case for DTMC is also similar. □

6. IMPLEMENTATION

We implemented our language in 1246 lines of Java, by defining its grammar and using ANTLR [1] to generate both parser and visitor components. Each syntax node within the abstract syntax tree (AST) was encapsulated in a corresponding `Node` class, with methods within these classes used for PRISM code generation.

```
String generateCode(ArrayList<Node> mods, int index, int maxIndex, boolean isCtmc, ArrayList<
    ↪ String> labels, String prot);
```

Listing 1: The `generateCode` function

The `generateCode` function generates the projection from our language to PRISM. The input parameters for the projection function include:

- **mods**: a list of the modules. New commands are appended to the set of commands for each respective module as they are generated.
- **index** and **maxIndex**: indices for tracking the current module being analyzed.
- **isCtmc**: a boolean flag indicating if a CTMC is being generated, crucial for projection generation logic.
- **labels**: existing labels; essential for checking label uniqueness.
- **prot**: the name of the protocol currently under analysis.

The projection function operates recursively on each command in the choreographic language, systematically generating PRISM code based on the type of command being analyzed. While most code generations are straightforward, the focal point lies in how new states are created. Each module maintains its set of states, and when a new state needs to be generated, the function examines the last available state for the corresponding module and increments it by one. Recursion follows a similar pattern: every module has as a field that accumulates recursion protocols, along with the first and last states associated with each recursion. This recursive approach ensures a systematic and coherent generation of states within the modules, improving the efficiency of the projection.

In Listing 2, we revisit the choreography from Example 2, assuming that process $\mathbf{P}$ executes the same branch for each process $\mathbf{Q}[i]$. Using our notation, we express the

commands concisely without repetition. For every module in the system, where the index i ranges from 1 to n , a corresponding PRISM module is generated. For instance, in the case where i ranges from 1 to 2, the resulting PRISM code is shown in Listing 3.

```

C := P → Q[i] : (+["mu1*gamma[i]" " (x'=1)" "(y[i] '=2)" ; C
                  +["mu2*gamma[i]" " (x'=3)" "(y[i] '=1)" ; C )

```

Listing 2: Example of an use of parameterization in the choreographic language

```

1  ctmc
2  module Q1
3    Q1.STATE : [0..1] init 0;
4    y1 : [0..N] init 0;
5    [RLICV] (Q1.STATE=0) → gamma1 : (y1'=2)&(Q1.STATE'=0);
6    [OKAMT] (Q1.STATE=0) → gamma1 : (y1'=1)&(Q1.STATE'=0);
7  endmodule
8
9  module Q2
10   Q2.STATE : [0..1] init 0;
11   y2 : [0..N] init 0;
12   [OMPXG] (Q2.STATE=0) → gamma2 : (y2'=2)&(Q2.STATE'=0);
13   [AQNZR] (Q2.STATE=0) → gamma2 : (y2'=1)&(Q2.STATE'=0);
14 endmodule
15
16 module P
17   P.STATE : [0..2] init 0;
18   x : [0..N] init 0;
19   [RLICV] (P.STATE=0) → mu1 : (x'=1)&(P.STATE'=0);
20   [OKAMT] (P.STATE=0) → mu2 : (x'=3)&(P.STATE'=0);
21   [OMPXG] (P.STATE=0) → mu1 : (x'=1)&(P.STATE'=0);
22   [AQNZR] (P.STATE=0) → mu2 : (x'=3)&(P.STATE'=0);
23 endmodule

```

Listing 3: PRISM code generated for the choreography in Listing 2

This modular approach systematically represents and integrates each system component within the PRISM framework, enabling comprehensive analysis and synthesis of the system's behavior. Importantly, these internal optimizations do not impact the projection process, as they focus on efficiency and code management rather than altering the overall structure or behavior of the projection.

The other differences are primarily syntactic. Updates of the same process are delineated by quotation marks, such as " $(x'=1)$ ". Additionally, rates and probabilities are represented differently. In our choreographic language the rate/probability of interaction is represented as the product of rates/probabilities of each process. For example, in Listing 2, we use $\text{mu1} * \text{gamma}[i]$ to indicate that the rate of the first process (P) is mu1 , while the rate of the second process ($Q[i]$) is represented by $\text{gamma}[i]$. If multiple processes are interacting, the rate/probability is the product of all corresponding rates/probabilities ($\text{lambda}_1 \dots \text{lambda}_n$).

In our implementation, we ensure a single enabled action per state by enforcing label uniqueness and unique state-associated variables. This clarity aids in accurately determining enabled actions and improves system reliability, facilitating analysis and comprehension of system dynamics.

7. BENCHMARKING

In this section, we present an experimental evaluation of our language. The examples provided highlight two main points: firstly, the representation using choreographic language is significantly more concise; secondly, we demonstrate that PRISM behaves similarly on both the projection and the original model in PRISM also in our implementation. In particular, we focus on six benchmarks: a modified version of the example reported in Section 2; a simple Peer-To-Peer protocol; the Bitcoin Proof of Work protocol [6]; the Hybrid Casper protocol [13]; a synchronous leader election protocol [17] and a modelization of the dining cryptographers [11]. The generated PRISM files can be found in our online repository [3].

A Modified thinkteam Protocol. In this modified version of the thinkteam protocol introduced in the earlier sections, we extend the protocol to involve generalised interactions with possible many receivers. Specifically, the **CheckOut** process now communicates with two users simultaneously, **User1** and **User2** each tasked with performing distinct actions upon access to the file.

In the first branch, **User1** increments the variable x by 1, while **User2** decrements the variable y by 1. Conversely, in the second branch, the roles are reversed, with **User1** decrementing x and **User2** incrementing y .

```

C0 := CheckOut → User1, User2 : (+["1*lambda"] " " "(x=x+1)" "(y=y-1)"; C1
                                +["1*lambda"] " " "(x=x-1)" "(y=y+1)"; C2)
C1 := CheckOut → User1, User2 : (+["1*theta"] ; C0)
C2 := CheckOut → User1, User2 : (+["1*mu"] ; C1 +["1*mu"] ; C2)

```

Listing 4: Choreography for the Modified thinkteam Protocol

Part of the generated PRISM model is reported in Listing 5.

```

1  module CheckOut
2    CheckOut.STATE : [0..2] init 0;
3    [MMHOL] (CheckOut.STATE=0) -> 1 : (CheckOut.STATE'=1);
4    [FFSFW] (CheckOut.STATE=0) -> 1 : (CheckOut.STATE'=2);
5    [ULCFN] (CheckOut.STATE=1) -> 1 : (CheckOut.STATE'=0);
6    [YHHWG] (CheckOut.STATE=2) -> 1 : (CheckOut.STATE'=1);
7    [XWSAO] (CheckOut.STATE=2) -> 1 : (CheckOut.STATE'=2);
8  endmodule
9  ...
10 module User2
11   User2.STATE : [0..2] init 0;
12   [MMHOL] (User2.STATE=0) -> lambda : (y'=y-1)&(User2.STATE'=1);
13   [FFSFW] (User2.STATE=0) -> lambda : (y'=y+1)&(User2.STATE'=2);
14   [ULCFN] (User2.STATE=1) -> mu : (User2.STATE'=0);
15   [YHHWG] (User2.STATE=2) -> theta : (User2.STATE'=1);
16   [XWSAO] (User2.STATE=2) -> theta : (User2.STATE'=2);
17 endmodule

```

Listing 5: Generated PRISM program

The generated PRISM model appears less clear compared to its choreographic representation, primarily due to its lack of sequential structure and lower readability. In the PRISM model, the absence of a clear sequential structure makes it harder to follow the flow of interactions between components, since module definitions and transition labels can be more challenging

to read and comprehend compared to the concise and structured nature of the choreographic language.

Simple Peer-To-Peer Protocol This case study describes a simple peer-to-peer protocol based on BitTorrent². The model comprises a set of clients trying to download a file that has been partitioned into K blocks. Initially, there is one client that has already obtained all of the blocks and N additional clients with no blocks. Each client can download a block from any of the others but they can only attempt four concurrent downloads for each block. The code we analyze with $K = 5$ and $N = 4$ is reported in Listing 6.

```
PeerToPeer := Client[i] → Client[i]:
    (+["rate1*1"] " (b[i]1'=1)"&&" " . PeerToPeer
    +["rate2*1"] " (b[i]2'=1)"&&" " . PeerToPeer
    +["rate3*1"] " (b[i]3'=1)"&&" " . PeerToPeer
    +["rate4*1"] " (b[i]4'=1)"&&" " . PeerToPeer
    +["rate5*1"] " (b[i]5'=1)"&&" " . PeerToPeer)
```

Listing 6: Choreography for the Peer-To-Peer Protocol

Part of the generated PRISM code is shown in Listing 7 and it is faithful with what is reported in the PRISM documentation.

In Figure 2, we compare the probabilities that all clients (in a model with 4 clients: **Client1**, ..., **Client4**) have received all blocks within the time interval $0 \leq T \leq 1.5$, as obtained from both our generated model and the model reported in the documentation. This property serves as a benchmark to evaluate whether the generated model preserves the expected behavior of the original specification. In this case, there are no differences in the results or the time required to compute the property.

```
1 module Client1
2   Client1 : [0..1] init 0;
3   b11 : [0..1];
4   b12 : [0..1];
5   b13 : [0..1];
6   b14 : [0..1];
7   b15 : [0..1];
8
9   [] (Client1=0) → rate1 : (b11'=1)&(Client1'=0);
10  [] (Client1=0) → rate2 : (b12'=1)&(Client1'=0);
11  [] (Client1=0) → rate3 : (b13'=1)&(Client1'=0);
12  [] (Client1=0) → rate4 : (b14'=1)&(Client1'=0);
13  [] (Client1=0) → rate5 : (b15'=1)&(Client1'=0);
14 endmodule
```

Listing 7: Generated PRISM program for the Peer-To-Peer Protocol

Proof of Work Bitcoin Protocol. In [6], the authors extended the PRISM model checker syntax to incorporate dynamic data types, enhancing its capabilities to model the Proof of Work protocol used in the Bitcoin blockchain [23].

```
PoW := Hasher[i] → Miner[i] :
    (+["mR*hr[i]" " " " (b[i]'=createB(b[i],B[i],c[i]))&(c[i]'=c[i]+1)" ;
    Miner[i] → Network : ("rB*1" " (B[i]'=addBlock(B[i],b[i]))")
```

²<https://www.prismmodelchecker.org/casestudies/peer2peer.php>

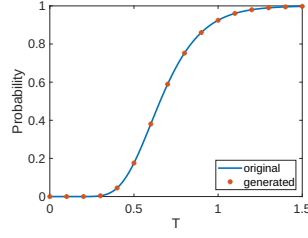


Figure 2: Probability that clients received all the block before T , with $0 \leq T \leq 1.5$

```

                                foreach(k!=i) "(set[k]'=addBlockSet(set[k],b[i]))"@Network;PoW)
+["lR*hR[i]" ] ;
  if "!isEmpty(set[i])"@Miner[i] then {
    ["r"] "(b[i]'=extractBlock(set[i]))"@Miner[i] ;
    Miner[i] → Network : ("1*1" "(setMiner[i]'=addBlockSet(setMiner[i],b[i]))"
                          "(set[i]' = removeBlock(set[i],b[i]))";PoW)
  }
  else{
    if "canBeInserted(B[i],b[i])"@Miner[i] then {
      ["1" " (B[i]'=addBlock(B[i],b[i]))&(setMiner[i]'=removeBlock(setMiner[i],b[i]))"@Miner[i];
      PoW
    }
    else{PoW}
  }
})

```

Listing 8: Choreography for the Proof of Work Bitcoin Protocol

In summary, the code depicts miners engaging in solving PoW, updating their ledgers, and communicating with the network. The indices i represent the module renaming feature of the choreographic language. Thus, each interaction will be repeated for each miner and hasher of the protocol that we are considering. The protocol works as follows. When synchronising with the hasher, a miner tries to solve a cryptographic puzzle. Successful attempts add a new block to its ledger and update other miners' block sets. Unsuccessful attempts involve extracting a block, updating its ledger and block sets, and continuing the PoW process.

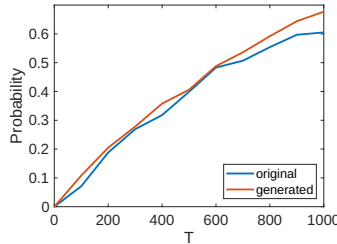


Figure 3: Probability that a block is created within T time units, $0 \leq T \leq 1000$

The PRISM model we created is more verbose than the one in [6], mainly because we consistently generate the else branch for if-then-else expressions, resulting in a higher number of instructions. Despite this, the experimental results for block creation probability within a bound time T (Figure 3) remain unaffected. Any discrepancies between the original

and generated models are due to inherent variations in the simulation-based calculation of probability.

Hybrid Casper Protocol. We now present the Hybrid Casper Protocol [13]. The Hybrid Casper protocol represents a hybrid consensus protocol for blockchains, merging features from both Proof of Work and Proof of Stake protocols.

```
PoS := Hasher[i] -> Validator[i] :
  (+["mR*1"] "(b[i]'=createB(b[i],L[i],c[i]))&(c[i]'=c[i]+1)";
    if "(mod(getHeight(b[i]),EpochSize)=0)"@Validator[i] then{...}
  else{
    Validator[i] -> Vote_Manager :(["1*1"] "(Votes'=addVote(Votes,b[i],stake[i]))"; PoS)}
  +["hR*1"] ; if "!isEmpty(set[i])"@Validator[i] then { ... }
    else{ PoS }
  +["rC*1"] "(lastCheck[i]'=extractCheckpoint(listCheckpoints[i],lastCheck[i]))"...
```

Listing 9: Excerpt of the Hybrid Casper Protocol as a choreography

The modeling approach is very similar to the one used for the Proof of Work Bitcoin protocol. Specifically, the Hybrid Casper protocol is represented in PRISM as the parallel composition of n `Validator` modules, along with the modules `Vote_Manager` and `Network`. Each `Validator` module closely resembles the `Miner` module from the previous protocol. The module `Vote_Manager` is responsible for storing maps containing votes for each block and computing associated rewards/penalties.

The choreographic model for this example is reported in Listing 9. The code resembles that of the Proof of Work protocol, but each validator can either create a new block, receive blocks from the network module, or determine if it's eligible to vote for specific blocks. For lack of space, we detailed only part of the code, the complete model can be found in [3].

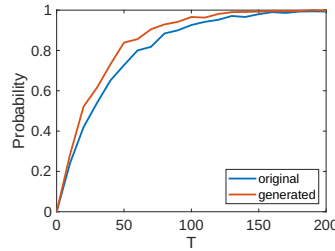


Figure 4: Probability that a block is created within T time units, $0 \leq T \leq 200$

The generated code is very similar the one outlined in [13], with the main distinction being the greater number of lines in our generated model. This difference is due to the fact that certain commands could be combined, but our generation lacks the automatic capability to perform this check. While the results obtained for the probability of creating a block within the time T reported in Figure 4 exhibit similarity, running simulations for the generated model takes PRISM 39.016 seconds, compared to the 22.051 seconds required for the original model.

Synchronous Leader Election. This case study examines the synchronous leader election protocol proposed by Itai & Rodeh [17], designed to elect a leader in a ring of N processors by exchanging messages. The protocol operates in rounds, where each processor selects a random ID from $\{1, \dots, K\}$, circulates it around the ring, and determines if a unique

maximum ID exists. If so, the processor with this ID becomes the leader; otherwise, the process repeats in the next round.

For illustration, we considered the case where $N = 4$ and $K = 8$, following the PRISM model³. We modeled this example in our choreographic language, as shown in Listing 10, capturing the protocol's behavior and dynamics.

```

Election := allSynch{ j in [1..4]
  Process[j] : (true -> "1/K" : "(p[i]'=0)&(v[i]'=0)&(u[i]'=true)" +
    ... + "1/K" : "(p[i]'=0)&(v[i]'=7)&(u[i]'=true)" ) }.

allSynch{
  Counter : ("(c<N-1)" -> "1" : "(c'=c+1)")
  Counter : ("(c=N-1)" -> "1" : "(c'=c)")
  Process1 : ("u1&!(p1=2)&(c<N-1)" -> "1" : "(u1'=true)&(v1'=v2)")
  Process1 : ("u1&(p1=2)&(c<N-1)" -> "1" : "(u1'=false)&(v1'=v2)&(p1'=0)")
  Process1 : ("!u1&(c<N-1)" -> "1" : "(u1'=false)&(v1'=v2)")
  Process1 : ("u1&!(p1=v2)&(c=N-1)" -> "1" : "(u1'=true)&(v1'=0)&(p1'=0)")
  Process1 : ("u1&(p1=v2)&(c=N-1)" -> "1" : "(u1'=false)&(v1'=0)&(p1'=0)")
  Process1 : ("!u1&(c=N-1)" -> "1" : "(u1'=false)&(v1'=0)")
  ...
}.
if "u1 | u2 | u3 | u4"@Counter then {
  Counter -> Process[i] : ([ "1*1" ] "(c'=c)" "(u[i]'=false)&(v[i]'=0)&(p[i]'=0)".
  allSynch {
    Counter : (true -> "1" : "(c'=c)")
    Process1 : (true -> "1" : " ")
    ...
  } . END)
}
else{ Counter -> Process[i] : ([ "1*1" ] "(c'=1)" "(u[i]'=false)&(v[i]'=0)&(p[i]'=0)" . Election)}

```

Listing 10: Choreography for the Synchronous Leader Election Protocol.

While the generated model (Listing 11) successfully replicates the functionality of the PRISM repository model, a key difference lies in the modular structure of the two representations. Specifically, the generated model adopts a simplified modular design by grouping transitions more compactly in certain modules, such as the **Counter** module. This simplification reduces redundancy and may improve readability without altering the correctness or outcomes of the protocol. Importantly, this structural refinement does not impact the behavior of the system, as the generated model remains functionally equivalent to the original PRISM repository model, as displayed in Figure 5.

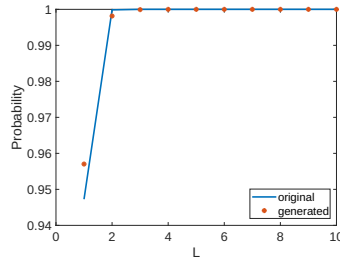


Figure 5: The probability of electing a leader within L rounds, with $1 \leq L \leq 10$

³https://www.prismmodelchecker.org/casestudies/synchronous_leader.php

```

1  module Counter
2      Counter : [0..4] init 0;
3      c : [0..N-1] init 0;
4      [YQBDX] (Counter = 0) & (c < N-1) -> 1 : (c' = c+1) & (Counter' = 1);
5      [YQBDX] (Counter = 0) & (c = N-1) -> 1 : (c' = c) & (Counter' = 1);
6      [ELTMI] (Counter = 1) & (u1 | u2 | u3 | u4) -> 1 : (c' = c) & (Counter' = 2);
7      [LJTIP] (Counter = 1) & ! (u1 | u2 | u3 | u4) -> 1 : (c' = 1) & (Counter' = 0);
8      [AWUQP] (Counter = 2) -> 1 : (c' = c) & (Counter' = 2);
9  endmodule
10 module Process1
11     Process1 : [0..4] init 0;
12     p1 : [0..K-1] init 0;
13     v1 : [0..K-1] init 0;
14     u1 : bool;
15     [BKKXT] (Process1 = 0) -> 1/K : (p1' = 0) & (v1' = 0) & (u1' = true) & (Process1' = 1)
16         + 1/K : (p1' = 1) & (v1' = 1) & (u1' = true) & (Process1' = 1)
17         + 1/K : (p1' = 2) & (v1' = 2) & (u1' = true) & (Process1' = 1)
18         + 1/K : (p1' = 2) & (v1' = 3) & (u1' = true) & (Process1' = 1)
19         + 1/K : (p1' = 2) & (v1' = 4) & (u1' = true) & (Process1' = 1)
20         + 1/K : (p1' = 2) & (v1' = 5) & (u1' = true) & (Process1' = 1)
21         + 1/K : (p1' = 2) & (v1' = 6) & (u1' = true) & (Process1' = 1)
22         + 1/K : (p1' = 2) & (v1' = 7) & (u1' = true) & (Process1' = 1);
23     [YQBDX] (Process1 = 1) & u1 & ! (p1 = 2) & (c < N-1) -> 1 : (u1' = true) & (v1' = v2) & (Process1' = 2);
24     [YQBDX] (Process1 = 1) & u1 & (p1 = 2) & (c < N-1) -> 1 : (u1' = false) & (v1' = v2) & (p1' = 0) & (Process1' = 2);
25     [YQBDX] (Process1 = 1) & ! u1 & (c < N-1) -> 1 : (u1' = false) & (v1' = v2) & (Process1' = 2);
26     [YQBDX] (Process1 = 1) & u1 & ! (p1 = v2) & (c = N-1) -> 1 : (u1' = true) & (v1' = 0) & (p1' = 0) & (Process1' = 2);
27     [YQBDX] (Process1 = 1) & u1 & (p1 = v2) & (c = N-1) -> 1 : (u1' = false) & (v1' = 0) & (p1' = 0) & (Process1' = 2);
28     [YQBDX] (Process1 = 1) & ! u1 & (c = N-1) -> 1 : (u1' = false) & (v1' = 0) & (Process1' = 2);
29     [ELTMI] (Process1 = 2) -> 1 : (u1' = false) & (v1' = 0) & (p1' = 0) & (Process1' = 3);
30     [LJTIP] (Process1 = 2) -> 1 : (u1' = false) & (v1' = 0) & (p1' = 0) & (Process1' = 0);
31     [AWUQP] (Process1 = 3) -> 1 : (Process1' = 4);
32 endmodule
33 ...

```

Listing 11: Generated PRISM program

Dining Cryptographers The generated model for this example does not faithfully model the original one. We chose to include it in the paper to demonstrate the limitations of our approach, specifically in cases where the abstraction may not fully capture the behavior of the original protocol. This allows us to analyze and understand where discrepancies may arise, highlighting areas for improvement in the model generation process. This case study explores the dining cryptographers protocol introduced by Chaum [11], which allows a group of N cryptographers to determine whether their master has anonymously paid for dinner without revealing the identity of the payer. The protocol functions by having each cryptographer flip a fair coin and share the outcome with their right-hand neighbor. Each cryptographer then publicly declares whether the two coins they observe—one they flipped and one received from the left—match or differ. If a cryptographer is the payer, they deliberately alter their response. The final count of “agree” statements follows a predictable pattern: for an odd number of cryptographers, an odd count indicates that one of them paid, while an even count means the master paid. This pattern reverses for an even number of participants.

To illustrate the protocol, we examined the case where $N = 3$, using the PRISM model⁴ reported in the official repository. We expressed this scenario in our choreographic language, as shown in Listing 12.

```

Crypto := if "(coin[i]=0)"@crypt[i] then {
    crypt[i] -> crypt[i] : (+["0.5*1"] "(coin[i]'=1)" . Crypto2
                        +["0.5*1"] "(coin[i]'=2)" . Crypto2)
}
else{ Crypto }

Crypto2 := if "((coin[i]>0)&(coin[i+1]>0))"@crypt[i] then{
    if "(coin[i]=coin[i+1])"@crypt[i] then {
        if "(pay=p[i])"@crypt[i] then { Crypto3 }
        else{ ["1"] "(agree[i]'=1)"@crypt[i]. Crypto3 }
    }
    else{
        if "(pay=p[i])"@crypt[i] then { ["1"] "(agree[i]'=1)"@crypt[i]. Crypto3 }
        else{ Crypto3 }
    }
}
else { Crypto2 }

Crypto3 := allSynch{ j in [1...3]
    crypt[j] : (true -> "1" : "true;" )
}.END

```

Listing 12: Choreography for the Dining Cryptographers Protocol.

This code defines a choreography model using three distinct choreographies (**Crypto**, **Crypto2**, and **Crypto3**) to avoid redundancy and optimize the code structure. By using separate choreographies, we can reuse common logic without rewriting the entire code. The initial probabilistic branching demonstrates the use of recursion, where both branches of the probabilistic transition ultimately lead to the same state.

In fact, in the generated PRISM model in Listing 13, we can observe that from the initial state **crypt1** = 0, we transition to two possible branches where **crypt1** = 1 regardless of whether **coin1** takes the value 1 or 2. This is a result of the recursion in the choreography, where both branches follow the same recursive path that leads to **crypt1** = 1. The transitions in the model are shown in the following PRISM code, where both transitions have a 50% probability of either setting **coin1** = 1 or **coin1** = 2, and both lead to the same updated state **crypt1** = 1. Note that the code provided here only shows a part of the generated PRISM model. The code for **crypt2** and **crypt3** follows the same structure.

```

1  ...
2  module crypt1
3  crypt1 : [0..2] init 0;
4  coin1 : [0..2] init 0;
5  s1 : [0..1] init 0;
6  agree1 : [0..1] init 0;
7  [] (crypt1=0)&((coin1=0)) -> 0.5 : (coin1'=1)&(crypt1'=1)+0.5 : (coin1'=2)&(crypt1'=1);
8  [] (crypt1=0)&!((coin1=0)) -> 1:(crypt1'=0);
9  [] (crypt1=1)&(coin1>0)&(coin2>0)&((coin1=coin2))&!((pay=p1)) -> 1:(agree1'=1)&(crypt1'=2);
10 [] (crypt1=1)&(coin1>0)&(coin2>0)&((coin1=coin2))&((pay=p1)) -> 1:(crypt1'=2);
11 [] (crypt1=1)&(coin1>0)&(coin2>0)&!((coin1=coin2))&((pay=p1)) -> 1:(agree1'=1)&(crypt1'=2);

```

⁴https://www.prismmodelchecker.org/casestudies/dining_cryptographers.php

```

12 [] (crypt1=1)&(coin1>0)&(coin2>0)&!((coin1=coin2))&!((pay=p1))-> 1:(crypt1'=2);
13 [LERZX] (crypt1 = 2) -> 1:(crypt1'=2);
14 endmodule
15 ...

```

Listing 13: Generated PRISM program

In this case, however, the generated model does not exhibit the same behavior as the original one in the PRISM repository. When analyzing the probability of anonymity, the result obtained by the original model, as reported on the website, is 0.25, whereas in our generated model, the probability is 0. This discrepancy arises from a fundamental difference in how the initial state is handled between the two models.

In the original PRISM model, the transition is defined as:

```

[] (coin1=0) -> 0.5 : (coin1'=1)&(crypt1'=1) + 0.5 : (coin1'=2)&(crypt1'=1);

```

Here, the transition occurs from `coin1 = 0` to `coin1 = 1` or `coin1 = 2`, without any dependency on the `crypt1` state. This allows for a transition regardless of the initial value of `crypt1`. In contrast, in our generated model, the transition is conditional on both `coin1 = 0` and `crypt1 = 0`, as shown at line 7 of Listing 13. As a result, the model can only transition when both conditions are met, but in the original model, such a restriction does not exist. This difference in the initial state setup explains why the probability of certain events and the number of states and transitions in our generated model differ significantly from that of the original model (95 states and 194 transitions in our model vs. 380 and 776 in the original model).

8. RELATED WORK AND DISCUSSION

Related Work. Choreographic programming [22] is a language paradigm for specifying the expected interactions (communications) of a distributed system from a global viewpoint, from which decentralised implementations can be generated via projection. The notion of choreography has been substantially explored in the last decade, both from a theoretical perspective, e.g., [8, 9], to full integration into fully-fledged programming languages, such as WS-CDL [16] and Choral [14]. Nevertheless, there is a scarcity of research on probabilistic aspects of choreographic programming. To the best of our knowledge, Aman and Ciobanu [4, 5] are the only ones who studied the concept of choreography and probabilities. Their work augments multiparty session types (type abstractions for communicating systems that use the concept of choreography) with a probabilistic internal choice similar to the one used by our choreographic branching. However, they do not provide any semantics with state in terms of Markov chains, and, most importantly, they do not project into a probabilistic declarative language model such as PRISM. Carbone et al. [7] define a logic for expressing properties of a session-typed choreography language. However, the logic is undecidable and has no model-checking algorithm. As far as our knowledge extends, there is currently no work that generates probabilistic models from choreographic languages that can be then model-checked.

Discussion and Future Work. The ultimate goal of the proposed framework is to use the concept of choreographic programming to improve several aspects, including usability, correctness, and efficiency in modeling and analysing systems. In this paper, we address usability and efficiency of modelling systems by proposing a probabilistic choreography

language. Our language improves the intuitive modeling of concurrent probabilistic systems. Traditional modeling languages often lack the expressive clarity needed to effectively capture the intricacies of such systems. By designing a language specific for choreographing system behaviors, we provide an intuitive means of specifying system dynamics. This approach enables a more natural and straightforward modeling process, essential for accurately representing real-world systems and ensuring the efficacy of subsequent analysis.

Although choreographies and the projection function aim to abstract away low-level details, providing instead a higher-level representation of system behaviors, the choreographic approach can have some limitations in expressivity. Some of the case studies presented in the PRISM documentation [2] cannot be modeled by using our current language. Specifically, there are two main cases where our approach encounters limitations: *(i)* in the asynchronous leader election case study, our language prohibits the use of an 'if-then' statement without an accompanying 'else' to prevent deadlocked states; *(ii)* in probabilistic broadcast protocols or cyclic server polling system models, the system requires probabilistic branching to synchronise different modules based on the selected branch. These issues could be fixed by extending our choreographic language further and are therefore left as future work.

Additionally to these extensions, we conjecture that our semantics for choreographies may be used for improving performance by directly generating a CTMC or a DTMC from a choreography, bypassing the projection into the PRISM language. In fact, the Markov chain construction from a choreography seems to be faster than the construction from a corresponding projection in the PRISM language, since it is not necessary to take into account all the possible synchronisations in the rules from Fig. 1. A formal complexity analysis, an implementation, and performance benchmarking are left as future work.

In conclusion, this paper has introduced a framework for addressing the challenges of modelling and analysing concurrent probabilistic systems. The development of a choreographic language with tailored syntax and semantics offers an intuitive modeling approach. We have established the correctness of a projection function that translates choreographic models to PRISM-compatible formats. Additionally, our compiler enables seamless translation of choreographic models to PRISM, facilitating powerful analysis while maintaining expressive clarity. These contributions bridge the gap between high-level modeling and robust analysis in probabilistic systems, paving the way for advancements in the field.

ACKNOWLEDGMENT

The authors wish to acknowledge fruitful discussions with A and B.

REFERENCES

- [1] ANTLR - another tool for language recognition. <https://www.antlr.org/>.
- [2] Prism documentation. <https://www.prismmodelchecker.org/>. Accessed: 2023-09-05.
- [3] Repository. <https://github.com/adeleveschetti/ChoreoPRISM>. Accessed: 2024-01-31.
- [4] Bogdan Aman and Gabriel Ciobanu. Probabilities in session types. In Mircea Marin and Adrian Craciun, editors, *Proceedings Third Symposium on Working Formal Methods, FROM 2019, Timișoara, Romania, 3-5 September 2019*, volume 303 of *EPTCS*, pages 92–106, 2019.
- [5] Bogdan Aman and Gabriel Ciobanu. Interval probability for sessions types. In Agata Ciabattini, Elaine Pimentel, and Ruy J. G. B. de Queiroz, editors, *Logic, Language, Information, and Computation - 28th International Workshop, WoLLIC 2022, Iași, Romania, September 20-23, 2022, Proceedings*, volume 13468 of *Lecture Notes in Computer Science*, pages 123–140. Springer, 2022.

- [6] Stefano Bistarelli, Rocco De Nicola, Letterio Galletta, Cosimo Laneve, Ivan Mercanti, and Adele Veschetti. Stochastic modeling and analysis of the bitcoin protocol in the presence of block communication delays. *Concurr. Comput. Pract. Exp.*, 35(16), 2023.
- [7] Marco Carbone, Davide Grohmann, Thomas T. Hildebrandt, and Hugo A. López. A logic for choreographies. In Kohei Honda and Alan Mycroft, editors, *Proceedings Third Workshop on Programming Language Approaches to Concurrency and communication-cEntric Software, PLACES 2010, Paphos, Cyprus, 21st March 2010*, volume 69 of *EPTCS*, pages 29–43, 2010.
- [8] Marco Carbone, Kohei Honda, and Nobuko Yoshida. Structured communication-centered programming for web services. *ACM Trans. Program. Lang. Syst.*, 34(2):8:1–8:78, 2012.
- [9] Marco Carbone and Fabrizio Montesi. Deadlock-freedom-by-design: multiparty asynchronous global programming. In Roberto Giacobazzi and Radhia Cousot, editors, *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, pages 263–274. ACM, 2013.
- [10] Marco Carbone and Adele Veschetti. A probabilistic choreography language for PRISM. In Ilaria Castellani and Francesco Tiezzi, editors, *Coordination Models and Languages - 26th IFIP WG 6.1 International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024, Groningen, The Netherlands, June 17-21, 2024, Proceedings*, volume 14676 of *Lecture Notes in Computer Science*, pages 20–37. Springer, 2024.
- [11] D. Chaum. The dining cryptographers problem: Unconditional sender and recipient untraceability. *Journal of Cryptology*, 1:65–75, 1988.
- [12] Frits Dannenberg, Marta Kwiatkowska, Chris Thachuk, and Andrew Turberfield. DNA walker circuits: computational potential, design, and verification. In D. Soloveichik and B. Yurke, editors, *Proc. 19th International Conference on DNA Computing and Molecular Programming (DNA 19)*, volume 8141 of *LNCS*, pages 31–45. Springer, 2013.
- [13] Letterio Galletta, Cosimo Laneve, Ivan Mercanti, and Adele Veschetti. Resilience of hybrid casper under varying values of parameters. *Distributed Ledger Technol. Res. Pract.*, 2(1):5:1–5:25, 2023.
- [14] Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti. Choral: Object-oriented choreographic programming. *ACM Trans. Program. Lang. Syst.*, 46(1):1:1–1:59, 2024.
- [15] J. Heath, M. Kwiatkowska, G. Norman, D. Parker, and O. Tymchyshyn. Probabilistic model checking of complex biological pathways. In C. Priami, editor, *Proc. Computational Methods in Systems Biology (CMSB'06)*, volume 4210 of *Lecture Notes in Bioinformatics*, pages 32–47. Springer Verlag, 2006.
- [16] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Web services, mobile processes and types. *Bull. EATCS*, 91:160–185, 2007.
- [17] A. Itai and M. Rodeh. Symmetry breaking in distributed networks. *Information and Computation*, 88(1), 1990.
- [18] M. Kwiatkowska, G. Norman, and D. Parker. Probabilistic verification of herman’s self-stabilisation algorithm. *Formal Aspects of Computing*, 24(4):661–670, 2012.
- [19] M. Kwiatkowska, G. Norman, D. Parker, and M.G. Vigliotti. Probabilistic mobile ambients. *Theoretical Computer Science*, 410(12–13):1272–1303, 2009.
- [20] M. Kwiatkowska, G. Norman, and R. Segala. Automated verification of a randomized distributed consensus protocol using Cadence SMV and PRISM. In G. Berry, H. Comon, and A. Finkel, editors, *Proc. 13th International Conference on Computer Aided Verification (CAV'01)*, volume 2102 of *LNCS*, pages 194–206. Springer, 2001.
- [21] M. Kwiatkowska, G. Norman, and J. Sproston. Probabilistic model checking of deadline properties in the IEEE 1394 FireWire root contention protocol. *Formal Aspects of Computing*, 14(3):295–318, 2003.
- [22] Fabrizio Montesi. *Introduction to Choreographies*. Cambridge University Press, 2023.
- [23] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- [24] G. Norman and V. Shmatikov. Analysis of probabilistic contract signing. *Journal of Computer Security*, 14(6):561–589, 2006.
- [25] Maurice H. ter Beek, Mieke Massink, Diego Latella, Stefania Gnesi, Alessandro Forghieri, and Maurizio Sebastianis. Model checking publish/subscribe notification for thinkteam®. In *FMICS*, volume 133 of *Electronic Notes in Theoretical Computer Science*, pages 275–294. Elsevier, 2004.